Retrieval-Augmented Generation (RAG) in LangGraph

Turning the multi-utility chatbot into a document-aware assistant: upload a PDF, retrieve the most relevant chunks, and answer grounded in your own content — wired in as just another tool.

WHAT THIS CHAPTER COVERS

- Why RAG: outdated knowledge, privacy, hallucination
- RAG architecture: split → embed → store → retrieve
- Loading and chunking a PDF
- VoyageAI embeddings and the FAISS vector store
- Building a retriever and wrapping it as a tool
- Assembling a standalone RAG chatbot graph
- Per-thread retriever isolation in the real project
- Wiring PDF upload into the Streamlit sidebar

Retrieval-Augmented Generation (RAG) in LangGraph

RECAP — CHAPTER 17

Chapter 17 connected the chatbot to external tool servers over MCP (local stdio + remote HTTP), solving the N×M tool-maintenance problem and wrapping everything in an async bridge so the sync LangGraph/Streamlit stack could call async MCP tools. This chapter adds a different kind of tool: one that lets the chatbot answer questions grounded in a document the user uploads, rather than only its own parametric knowledge.

1. Why RAG Exists

Definition

Retrieval-Augmented Generation (RAG) is a technique where relevant external content is retrieved and injected into an LLM's prompt as context, so the model can answer questions it could never answer from training alone.

Why It Exists — Three Problems

Problem	What Goes Wrong	How RAG Helps
Outdated knowledge	Every LLM has a training/knowledge cut-off date; anything after it is unknown to the model.	Retrieve current information and pass it in as context at query time.
Privacy	Private documents (company financials, personal expense sheets) were never part of any model's training data.	Connect the LLM to your own private documents without retraining it.
Hallucination	An LLM can confidently generate false information (e.g. fabricated paper links that 404).	Ground the response in retrieved, verifiable context instead of letting the model invent facts.

REAL WORLD INSIGHT

A general-purpose chat product answering questions about “today’s news” is conceptually doing RAG behind the scenes — it runs a web search, pulls back real content, and generates a response grounded in that content.

> ↻ **Quick Revision — Why RAG Exists**
>
> • RAG = retrieve relevant content, then feed it to the LLM as context.
>
> • Three driving problems: outdated knowledge, privacy, hallucination.
>
> • The most common real-world use case: chatting with your own private documents.

2. RAG Architecture at a Glance

The Core Principle: In-Context Learning

RAG works on a simple principle: if you provide an LLM with additional context at chat time, it can answer using that context in combination with its own parametric knowledge (what it learned during training). The naive version of this — pasting an entire document into the prompt — breaks down quickly because every LLM has a **limited context window** (a maximum number of tokens it can process). A single expense sheet might fit; a folder of 100 e-books, or an entire company codebase, will not.

Why Context Must Be Filtered

Instead of pasting the full source as-is, RAG pastes only the part of the context that is relevant to the current query — for example, 5–6 pages out of a 200-page book, not the whole book. This filtering step is what makes RAG scale to large knowledge sources.

How It Works — Ingestion and Query-Time

1. **Split** the knowledge source (a book, a PDF, a webpage) into smaller chunks.
2. **Embed** each chunk into a vector using an embedding model — a set of numbers that captures the chunk's semantic meaning.
3. **Store** these embeddings (plus the original text) in a specialized database called a *vector store* (e.g. FAISS, Chroma).
4. At query time, the user's question is embedded the same way, and a *retriever* compares that query vector against the stored vectors to find the closest (most semantically similar) chunks.
5. The retrieved chunks' text is combined with the original query into a single prompt and sent to the LLM, which answers using both the context and its own parametric knowledge.

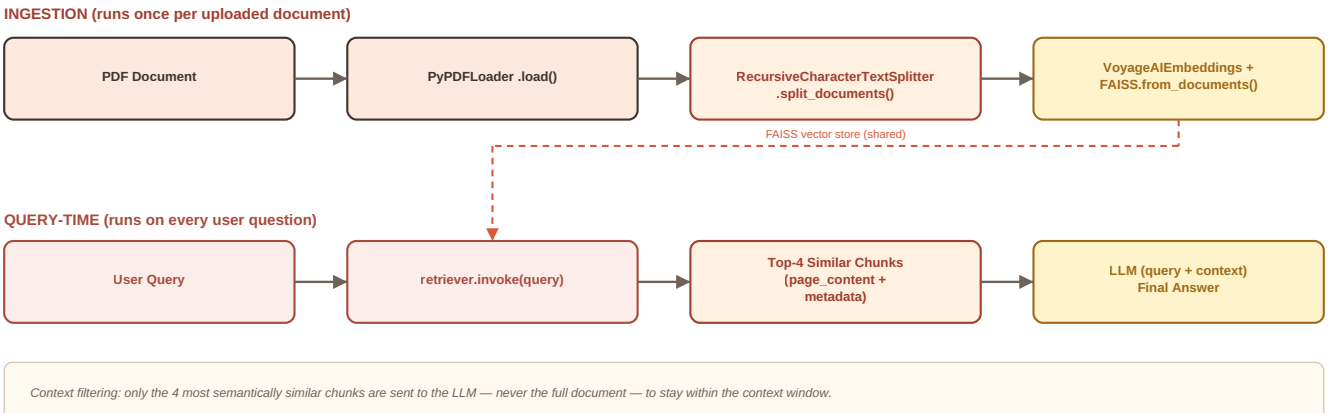


Figure 18.1 — Ingestion builds the FAISS vector store once; query-time retrieval runs on every question and converges on that same store.

↻ Quick Revision — RAG Architecture

- In-context learning: give the LLM extra context at chat time instead of relying only on its training.
- Context windows are limited, so the full source is never pasted in — only filtered, relevant chunks are.
- Pipeline: split → embed → store (vector store) → retrieve → combine with query → LLM answer.
- A retriever converts the query to an embedding, then finds the closest stored vectors by semantic similarity.

3. Standalone RAG Notebook — Setup

Before integrating RAG into the existing multi-utility chatbot, the concept is first built from scratch in a separate notebook (`rag.ipynb`). This mirrors the approach used for every new LangGraph concept in this series: prove it in isolation first, then fold it into the running project.

Imports and LLM

```
from langchain_groq import ChatGroq
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_core.tools import tool
from langchain_voyageai import VoyageAIEmbeddings
from langgraph.prebuilt import ToolNode, tools_condition

llm = ChatGroq(model = 'openai/gpt-oss-safeguard-20b')
```

COMMON MISTAKE

The lecture narration describes using `gpt-4o-mini` and an OpenAI embeddings model (`text-embedding-3-small`) as illustrative examples. The actual attached notebook and backend code use **ChatGroq** (`openai/gpt-oss-safeguard-20b`) for the LLM and **VoyageAIEmbeddings** (`voyage-3-large`) for embeddings. Per this project's source hierarchy, attached code is authoritative — these notes follow the code, not the spoken example.

Loading the Document

The first step of ingestion is loading the raw PDF into LangChain `Document` objects using `PyPDFLoader`, one `Document` per page.

```
loader = PyPDFLoader('intro-to-ml.pdf')
docs = loader.load()
```

↻ Quick Revision — Setup

- LLM: ChatGroq with model 'openai/gpt-oss-safeguard-20b' (per attached code, not the lecture's spoken gpt-4o-mini example).
- PyPDFLoader('intro-to-ml.pdf').load() returns one Document per PDF page.

4. Splitting the Document into Chunks

Why It Exists

A raw page-level Document is often too large or too coarse a retrieval unit. `RecursiveCharacterTextSplitter` breaks each document into smaller, more retrieval-friendly chunks.

Implementation

```
splitter = RecursiveCharacterTextSplitter(chunk_size = 1000, chunk_overlap = 200)
chunks = splitter.split_documents(docs)
```

KEY PARAMETERS

- **chunk_size** — the maximum number of characters per chunk (1000 here).
- **chunk_overlap** — how many characters consecutive chunks share (200 here), so context is not lost right at a chunk boundary.

BEST PRACTICE

Keep some overlap between chunks. Without it, a sentence or idea that straddles a chunk boundary can be split in half, and neither resulting chunk fully captures its meaning — hurting retrieval quality for queries about that content.

↻ Quick Revision — Splitting

- RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200) is the splitter used in this project.
- chunk_overlap preserves context across chunk boundaries.
- .split_documents(docs) returns the list of chunks passed on to the embedding step.

5. Embeddings and the Vector Store

Definition

An *embedding model* converts text into a vector (a list of numbers) that captures its semantic meaning. Chunks with similar meaning end up with vectors that are close together in that vector space.

Implementation

```
embedding_model = VoyageAIEmbeddings(
    model = "voyage-3-large"
)
vector_store = FAISS.from_documents(chunks, embedding_model)
vector_store.save_local("my_local_faiss_index")
```

`FAISS.from_documents()` runs the embedding model over every chunk and stores the resulting vectors — alongside each chunk's original text and metadata — inside a **FAISS** vector store (a similarity-search library originally built by Facebook/Meta). `save_local()` persists the index to disk so it does not need to be rebuilt on every run.

> **IMPORTANT**
>
> A vector store does not just store vectors. It stores the vector *and* its corresponding original text (and metadata), so that once a similar vector is found, the actual readable content behind it can be retrieved and handed to the LLM.

↻ Quick Revision — Embeddings and Vector Store

- VoyageAIEmbeddings('voyage-3-large') turns each chunk into a semantic vector.
- FAISS.from_documents(chunks, embedding_model) builds the vector store; save_local() persists it to disk.
- Each stored vector is paired with its source chunk's text and metadata.

6. Building the Retriever and Wrapping It as a Tool

Creating the Retriever

```
retriever = vector_store.as_retriever(search_type = "similarity", search_kwargs = {'k':4})
```

This builds a `retriever` object configured for similarity search, returning the top `k=4` most similar chunks for any query. Calling `retriever.invoke(query)` embeds the query, compares it against every stored vector, and returns the 4 closest `Document` objects (each with `page_content` and `metadata`).

Wrapping the Retriever as a Tool

Rather than inventing a new mechanism for RAG, this project treats retrieval as **just another tool** — exactly like the calculator or stock-price tools from Chapter 16. This is a reusable template: in virtually any agentic AI application, RAG is implemented as a tool the LLM can choose to call.

```
@tool
def rag_tool(query):
    """
    Retrieve relevant information from the pdf document.
    Use this tool when user asks factual/conceptual questions
    that might be answered from the stored documents.
    """

    result = retriever.invoke(query)

    context = [doc.page_content for doc in result]
    metadata = [doc.metadata for doc in result]

    return {
        'query':query,
        'context':context,
        'metadata':metadata
    }
```

> **IMPORTANT**
>
> The tool's docstring is not documentation for humans — it is what the LLM reads to decide *when* to call `rag_tool`. A vague or missing docstring means the model will not reliably reach for RAG even when a document has been uploaded.

↻ Quick Revision — Retriever and Tool

- retriever = vector_store.as_retriever(search_type='similarity', search_kwargs={'k':4}) returns the top-4 similar chunks per query.
- rag_tool wraps retriever.invoke() behind the @tool decorator, returning query + context + metadata.
- RAG is implemented as an ordinary LangGraph tool, not a separate mechanism.

7. Assembling the Standalone RAG Chatbot

Binding the Tool and Defining State

```
tools = [rag_tool]
llm_with_tools = llm.bind_tools(tools)

class ChatState(TypedDict):
    messages : Annotated[List[BaseMessage], add_messages]
```

Nodes and Graph

```
def chat_node(state: ChatState):
    messages = state['messages']
    response = llm_with_tools.invoke(messages)
    return {'messages':[response]}

tool_node = ToolNode(tools)

graph = StateGraph(ChatState)
graph.add_node('chat_node', chat_node)
graph.add_node('tools', tool_node)

graph.add_edge(START, 'chat_node')
graph.add_conditional_edges('chat_node', tools_condition)
graph.add_edge('tools', 'chat_node')

chatbot = graph.compile()
```

This is the same tool-calling loop pattern established in Chapter 16: `tools_condition` decides whether the model needs a tool, and the single feedback edge `tools → chat_node` lets the model turn retrieved context into a final answer.

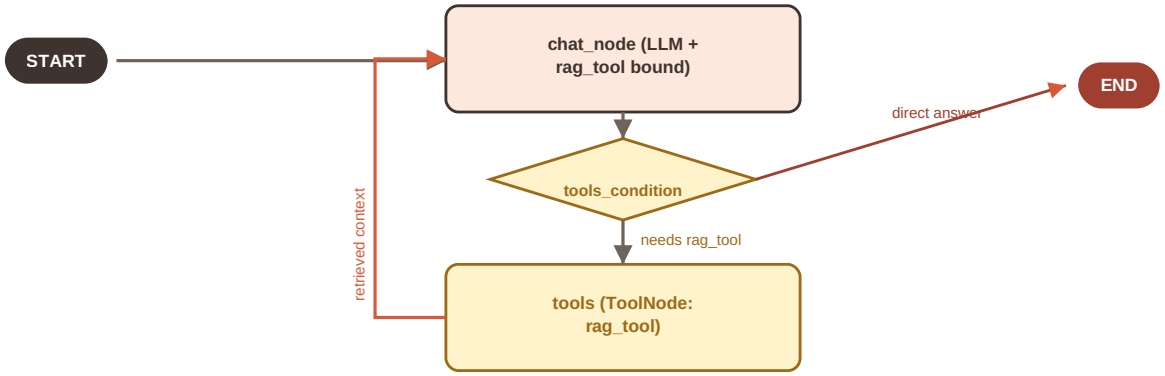


Figure 18.2 — rag_tool is wired exactly like any other tool: chat_node decides, tools executes, and the loop feeds retrieved context back for a final answer.

> ↻ **Quick Revision — Graph Assembly**
>
> • ChatState holds only messages, with the add_messages reducer.
>
> • tools_condition routes to the tools node only when the model's response contains a tool call.
>
> • The single tools -> chat_node edge closes the loop so retrieved context reaches the final answer.

8. Running the Standalone RAG Chatbot

Example

```
result = chatbot.invoke(
    {
        'messages':[
            HumanMessage(
                content = ('Using the pdf notes, explain how to split a node in decision
tree?')
            )
        ]
    }
)

print(result['messages'][-1].content)
```

Execution Flow

1. The question reaches `chat_node`; the LLM decides it needs `rag_tool` and requests a call with the query text.

2. Control passes to the `tools` node, which runs `rag_tool`, searching the FAISS store for the 4 most similar chunks about splitting a node in a decision tree.
3. Those chunks flow back to `chat_node` as a tool result; the LLM now has the original question *and* the retrieved context.
4. `chat_node` combines both to generate the final, document-grounded answer.

> **REAL WORLD INSIGHT**
>
> Each document-grounded query takes a noticeable amount of time (roughly 8–9 seconds in the lecture's demo) because it involves a round trip: embed the query, search the vector store, retrieve chunks, then run the LLM generation step — versus a single LLM call for a non-document question.

↻ Quick Revision — Running the Chatbot

- chatbot.invoke() with a single HumanMessage triggers the full retrieve-then-answer flow.
- result['messages'][-1].content holds the final, grounded answer.
- A document query is slower than a plain chat turn because it includes a retrieval round trip.

9. Integrating RAG into the Multi-Utility Chatbot

The Integration Challenge

The standalone notebook uses one global `retriever` tied to a single hardcoded PDF. The real project supports many concurrent chat threads, and each thread's user may upload a *different* PDF (or none at all). The integration therefore needs a **per-thread retriever store** instead of one shared retriever.

ingest_pdf() and Per-Thread Storage

```
_THREAD_RETRIEVERS: Dict[str, Any] = {}
_THREAD_METADATA: Dict[str, dict] = {}

def ingest_pdf(file_bytes: bytes, thread_id: str, filename: Optional[str] = None) -> dict:
    loader = PyPDFLoader(temp_path)
    docs = loader.load()

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000, chunk_overlap=200, separators=["\n\n", "\n", " ", ""]
    )
    chunks = splitter.split_documents(docs)

    vector_store = FAISS.from_documents(chunks, embeddings)
    retriever = vector_store.as_retriever(
        search_type="similarity", search_kwargs={"k": 4}
    )

    _THREAD_RETRIEVERS[str(thread_id)] = retriever
    _THREAD_METADATA[str(thread_id)] = {
        "filename": filename, "documents": len(docs), "chunks": len(chunks),
    }
```

The uploaded PDF's bytes are written to a temp file, loaded, split, embedded, and stored exactly as in the standalone notebook — but the resulting retriever is saved into `_THREAD_RETRIEVERS` keyed by `thread_id`, so each chat thread's document stays isolated from every other thread's.

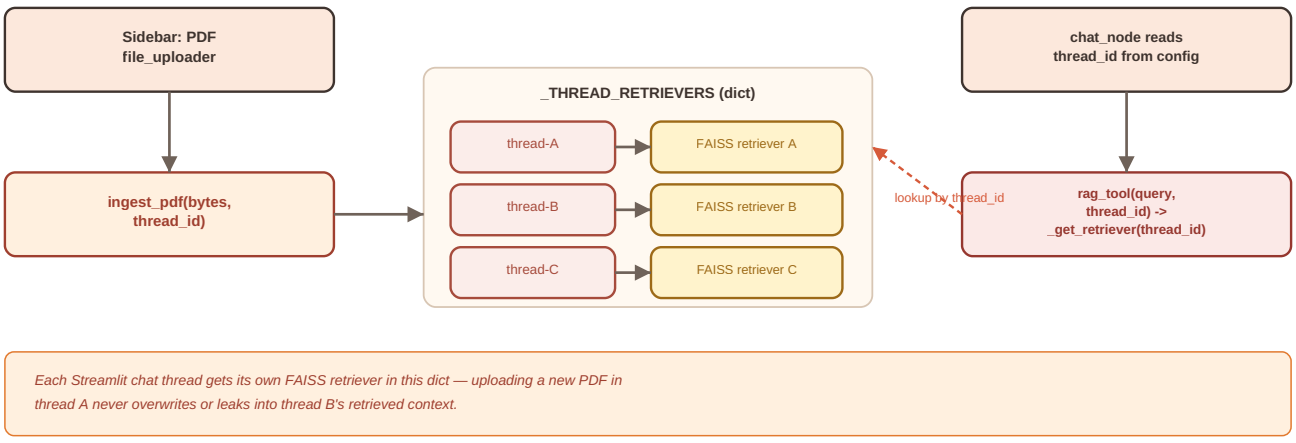


Figure 18.3 — Each Streamlit thread gets its own FAISS retriever in a thread_id-keyed dict; chat_node and rag_tool both read thread_id from the running config.

rag_tool Gains a thread_id Parameter

```
@tool
def rag_tool(query: str, thread_id: Optional[str] = None) -> dict:
    """
    Retrieve relevant information from the uploaded PDF for this chat thread.
    Always include the thread_id when calling this tool.
    """
    retriever = _get_retriever(thread_id)
    if retriever is None:
        return {"error": "No document indexed for this chat. Upload a PDF first.", "query":
query}

    result = retriever.invoke(query)
    return {
        "query": query,
        "context": [doc.page_content for doc in result],
        "metadata": [doc.metadata for doc in result],
        "source_file": _THREAD_METADATA.get(str(thread_id), {}).get("filename"),
    }
```

Because the LLM has no built-in notion of which Streamlit thread it is running inside, `chat_node` reads `thread_id` out of the LangGraph `config` and places it directly in the system message, instructing the model to pass that exact `thread_id` whenever it calls `rag_tool`:

```
system_message = SystemMessage(
    content=(
        "You are a helpful assistant. For questions about the uploaded PDF, call "
        "the `rag_tool` and include the thread_id "
        f"`{thread_id}`. You can also use the web search, stock price, and "
        "calculator tools when helpful. If no document is available, ask the user "
        "to upload a PDF."
    )
)
```

INTERVIEW TIP

Be ready to explain why thread_id is pushed into the system prompt text rather than read silently inside rag_tool: the tool function itself has no access to the LangGraph run config, only to whatever arguments the LLM supplies in its tool call — so the model has to be told, in plain language, to pass it along.

COMMON MISTAKE

Forgetting to pass `config` through to `llm_with_tools.invoke(messages, config=config)` inside `chat_node` would mean `config.get("configurable", {}).get("thread_id")` is never available to build that system message in the first place, silently breaking the per-thread document lookup.

↻ Quick Revision — Integration

- ingest_pdf(file_bytes, thread_id, filename) builds and stores a FAISS retriever per Streamlit thread.
- _THREAD_RETRIEVERS and _THREAD_METADATA are dicts keyed by str(thread_id).
- rag_tool gained a thread_id parameter; chat_node injects the current thread_id into the system message so the LLM remembers to pass it.
- Embeddings model (VoyageAIEmbeddings 'voyage-3-large') is created once at module load and reused for every thread's ingestion.

10. Frontend: PDF Upload in the Sidebar

Sidebar File Uploader

```
uploaded_pdf = st.sidebar.file_uploader("Upload a PDF for this chat", type=["pdf"])
if uploaded_pdf:
    if uploaded_pdf.name in thread_docs:
        st.sidebar.info(f"`{uploaded_pdf.name}` already processed for this chat.")
    else:
        with st.sidebar.status("Indexing PDF…", expanded=True) as status_box:
            summary = ingest_pdf(
                uploaded_pdf.getvalue(),
                thread_id=thread_key,
                filename=uploaded_pdf.name,
            )
            thread_docs[uploaded_pdf.name] = summary
            status_box.update(label="✅ PDF indexed", state="complete", expanded=False)
```

`st.sidebar.status(...)` gives the user a visible "Indexing PDF…" progress indicator while `ingest_pdf()` runs (load → split → embed → store), then flips to a completed, collapsed state once indexing finishes. A simple membership check (`uploaded_pdf.name in thread_docs`) avoids re-indexing the same file twice for the same thread.

Surfacing Which Document Is Active

```
if thread_docs:
    latest_doc = list(thread_docs.values())[-1]
    st.sidebar.success(
        f"Using `{latest_doc.get('filename')}` "
        f"({latest_doc.get('chunks')} chunks from {latest_doc.get('documents')} pages)"
    )
else:
    st.sidebar.info("No PDF indexed yet.")
```

After a chat turn, the frontend also surfaces the active document's metadata as a caption under the answer, using `thread_document_metadata(thread_key)` — so the user always knows which file (and how many chunks/pages) the chatbot is grounding its RAG answers in.

BEST PRACTICE

Surfacing chunk and page counts in the UI (not just a filename) gives the user a quick sanity check that ingestion actually completed — a 0-chunk result is an immediate, visible signal that something went wrong with the upload.

↻ Quick Revision — Frontend

- st.sidebar.file_uploader(type=['pdf']) plus a thread-scoped membership check avoids duplicate indexing.
- st.sidebar.status(...) shows live 'Indexing PDF...' progress around the ingest_pdf() call.
- thread_document_metadata(thread_key) drives both the sidebar summary and a per-answer caption.

K. Key Takeaways

- **RAG** grounds LLM answers in retrieved external content, solving outdated-knowledge, privacy, and hallucination problems.
- **In-context learning** is the underlying principle: give the LLM extra, relevant context at chat time instead of relying only on training data.
- **The RAG pipeline** is split → embed → store (vector store) → retrieve → combine with query → LLM answer.
- **RecursiveCharacterTextSplitter** (chunk_size=1000, chunk_overlap=200) breaks a document into retrieval-friendly, overlapping chunks.
- **VoyageAIEmbeddings + FAISS** turn chunks into vectors and store them for fast similarity search.
- **rag_tool** wraps retriever.invoke() behind the @tool decorator — RAG is implemented as just another LangGraph tool.
- **The standalone graph** reuses the Chapter 16 tool-loop pattern: chat_node → tools_condition → tools → chat_node.
- **Per-thread retriever isolation** (_THREAD_RETRIEVERS keyed by thread_id) lets every Streamlit chat thread index its own PDF independently.
- **thread_id propagation** happens via the system message, since rag_tool itself cannot read the LangGraph run config directly.

R. Revision Sheet

RAG

Retrieve relevant external content and inject it into the LLM prompt as context before generating an answer.

Embedding

A vector representation of text that captures its semantic meaning, produced by an embedding model.

Vector Store

A specialized database (e.g. FAISS) that stores embeddings alongside their source text/metadata for similarity search.

Retriever

Converts a query into a vector and returns the top-k most similar chunks from a vector store.

chunk_size / chunk_overlap

Splitter parameters controlling chunk length and how much consecutive chunks overlap to preserve context.

rag_tool

An @tool-decorated function wrapping retriever.invoke(), returning query, context, and metadata to the LLM.

Per-Thread Isolation

_THREAD_RETRIEVERS dict keyed by thread_id, so each chat thread's uploaded PDF stays independent of others.

thread_id Injection

chat_node reads thread_id from config and embeds it in the system message so the LLM passes it to rag_tool.

I. Interview Questions

Q1. What three problems does RAG solve, and why can't an LLM solve them from its parametric knowledge alone?

Q2. Explain the principle of in-context learning and how it relates to RAG.

Q3. Why can't you simply paste an entire document into an LLM prompt, and what does RAG do instead?

Q4. Walk through the RAG pipeline from a raw PDF to a final grounded answer.

Q5. What do chunk_size and chunk_overlap control, and why does overlap matter?

Q6. What does a vector store actually store, besides the embeddings themselves?

Q7. Why is RAG implemented as a LangGraph tool rather than as a separate custom node type?

Q8. Why does a well-written tool docstring matter for a tool like rag_tool?

Q9. How does this project isolate one chat thread's uploaded PDF from another thread's?

Q10. Why does rag_tool need an explicit thread_id parameter instead of silently reading it from LangGraph's run config?

F. Further Reading

- LangChain documentation — Retrieval-Augmented Generation (RAG) concepts and tutorials
- LangChain documentation — Text splitters (RecursiveCharacterTextSplitter)
- FAISS (Facebook AI Similarity Search) — official documentation and GitHub repository
- VoyageAI — embeddings model documentation
- LangGraph documentation — Tool calling and the prebuilt ToolNode / tools_condition